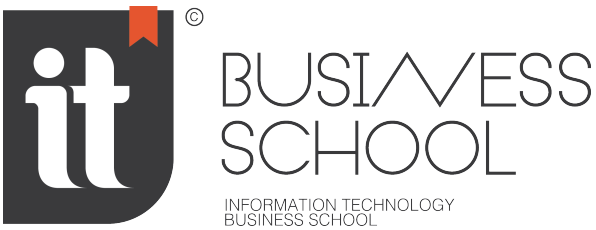


République Tunisienne

Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

École Supérieure Privée des Technologies de l'Information et de Management de Nabeul



Réalisé par

Imen Sioua

Document d'analyse fonctionnelle

LogWatch

Application de monitoring et d'analyse de logs
Stack ELK, NoSQL et interface web

Encadrant académique :

M. Mohamed Najeh Issaoui

Année universitaire 2026-2027

Partie A — Document d'analyse fonctionnelle

A.1 Contexte et scénario choisi

Dans un système d'information distribué, les applications, les serveurs et les bases de données produisent un très grand volume de logs. Pour une application SaaS utilisée par des milliers d'entreprises clientes, ces logs sont dispersés sur plusieurs serveurs : le diagnostic des incidents est long, aucune alerte automatique n'existe et les tendances ne sont pas visualisées.

Mission : concevoir, développer et déployer LogWatch, une plateforme qui centralise la collecte des logs, les indexe pour une recherche rapide, propose des visualisations et une interface web, et se déploie par conteneurs (Docker Compose).

Problématique

Comment centraliser et analyser efficacement les logs d'une application SaaS afin de détecter rapidement les erreurs, identifier les problèmes de performance et faciliter la prise de décision par les équipes techniques et métier ?

Périmètre du projet

Le périmètre de LogWatch couvre la collecte, l'indexation, la recherche et la visualisation des logs d'une application SaaS, ainsi que le suivi des principaux indicateurs de performance.

Les fonctionnalités d'authentification, de cache, d'alerting et d'observabilité sont prévues selon les priorités définies dans le Product Backlog.

Scénario choisi : D — Application web SaaS

Types de logs traités :
- Logs serveur web (Nginx / Apache : accès, erreurs)
- Logs applicatifs (Django / Flask : exceptions, warnings, debug)
- Logs de base de données (requêtes lentes, deadlocks, erreurs)
- Logs de performances (temps de réponse, mémoire / CPU)
- Logs d'utilisation métier (features utilisées, actions des utilisateurs)

Indicateurs clés (KPI) à suivre :
- Temps de réponse moyen par endpoint
- Taux d'erreur 5xx par service
- Requêtes SQL les plus lentes
- Nombre d'utilisateurs actifs simultanés
- Features les plus et les moins utilisées

Cas d'usage prioritaires : alerter l'équipe DevOps si le temps de réponse dépasse 3 secondes ; identifier les requêtes SQL à optimiser ; analyser l'adoption des nouvelles fonctionnalités.

Justification du choix

— Les logs d'une application SaaS sont simples à produire et à comprendre (accès web, erreurs, requêtes SQL lentes).
— Les KPI sont concrets et directement démontrables dans Kibana et dans l'interface web.
— Le cas prioritaire « alerte au-delà de 3 secondes » se prête bien au module avancé d'alerting.

En clair : on a choisi les logs d'une application SaaS parce que les problèmes à détecter (lenteurs, erreurs 5xx) sont faciles à expliquer et à montrer.

A.2 Acteurs du système

En clair : les acteurs sont les personnes ou les systèmes extérieurs qui interagissent avec l'application.

Acteur	Type	Rôle dans le système
Ingénieur DevOps / Support technique	Humain (équipe technique)	Téléverse les fichiers de logs, suit leur ingestion, recherche et diagnostique les incidents, gère les alertes, surveille la santé de la plateforme
Analyste métier	Humain (équipe métier)	Consulte le dashboard principal, les dashboards Kibana et les métriques business
Administrateur	Humain	Hérite des droits du DevOps ; gère les utilisateurs et les rôles
SMTP / Slack / Discord / Teams	Système externe (avancé)	Reçoivent les notifications d'alerte (email et webhook)

A.3 Besoins fonctionnels et non fonctionnels

Besoins fonctionnels

ID	Besoin	Module du cahier	Prio.
BF1	Uploader des fichiers CSV, JSON, TXT (drag & drop, sélection du type de fichier, validation côté client, taille maximale configurable, barre de progression)	Modules 1 et 3	P1
BF2	Valider les fichiers et prévisualiser les 10 premières lignes	Module 1	P1
BF3	Injecter automatiquement les fichiers dans Logstash et suivre leur statut	Module 1	P1
BF4	Lister les fichiers uploadés (tri, pagination, filtres)	Module 1	P1
BF5	Indexer les logs : index template Elasticsearch et 2 pipelines Logstash (CSV, JSON)	Module 2	P1
BF6	Visualiser les logs : 3 visualisations Kibana (temps de réponse par endpoint, erreurs 5xx, utilisateurs actifs), les visualisations des requêtes SQL les plus lentes et des features les plus et les moins utilisées, et 1 dashboard exportable en JSON	Module 2	P1

ID	Besoin	Module du cahier	Prio.
BF7	Afficher un dashboard principal (statistiques globales, graphiques, tableau des derniers logs)	Module 3	P1
BF8	Rechercher en texte libre avec filtres niveau, service et dates	Module 3	P1
BF9	Afficher les résultats (50 par page, tri, détail, export CSV)	Module 3	P1
BF10	Intégrer Kibana dans l'application web (page dédiée accessible depuis le menu)	Module 3	P1
BF11	Offrir une interface responsive avec navigation claire (navbar, footer, fil d'Ariane, messages flash, page 404 personnalisée)	Module 3	P1
BF12	Stocker dans MongoDB les métadonnées des fichiers et l'historique des recherches, avec une page d'historique	Module 4	P1
BF13	Déployer toute la plateforme avec Docker Compose et la configurer par un fichier .env	Module 5	P1
BF14	Authentification et gestion des rôles	Intermédiaire	P2
BF15	Cache Redis des résultats de recherche et limitation de débit de l'API	Intermédiaire	P2
BF16	API REST documentée avec Swagger	Intermédiaire	P2
BF17	Alerting : créer et tester des règles d'alerte, worker (silencing, cooldown), notifications email et webhook, dashboard des alertes avec actions (acquitter, résoudre, suspendre) et commentaires, escalade automatique	Module G	P3
BF18	Observabilité : /health, logs JSON, Prometheus, Grafana, métriques business, alertes d'infrastructure, tracing	Module L	P3

Besoins non fonctionnels

— **Performance :** les logs d'un fichier uploadé sont consultables dans Elasticsearch environ 10 secondes après l'upload ; les résultats de recherche sont paginés par 50.
— **Capacité :** fichiers jusqu'à 100 Mo (limite configurable).
— **Portabilité :** toute la plateforme démarre avec `docker compose up`, y compris sur une machine vierge.
— **Sécurité :** secrets dans un fichier `.env` non versionné ; mots de passe hachés ; pages protégées par authentification ; limitation du nombre de requêtes API (rate limiting).
— **Ergonomie :** interface responsive (desktop 1920×1080 minimum, tablette, mobile), testée sur Chrome, Firefox et Safari.
— **Fiabilité :** health checks Docker, messages d'erreur clairs, gestion des erreurs de parsing et de connexion, journalisation applicative.
— **Qualité et maintenabilité :** tests unitaires et d'intégration, code commenté, README complet, API documentée avec Swagger.

A.4 Diagramme de cas d'utilisation (UML Use Case)

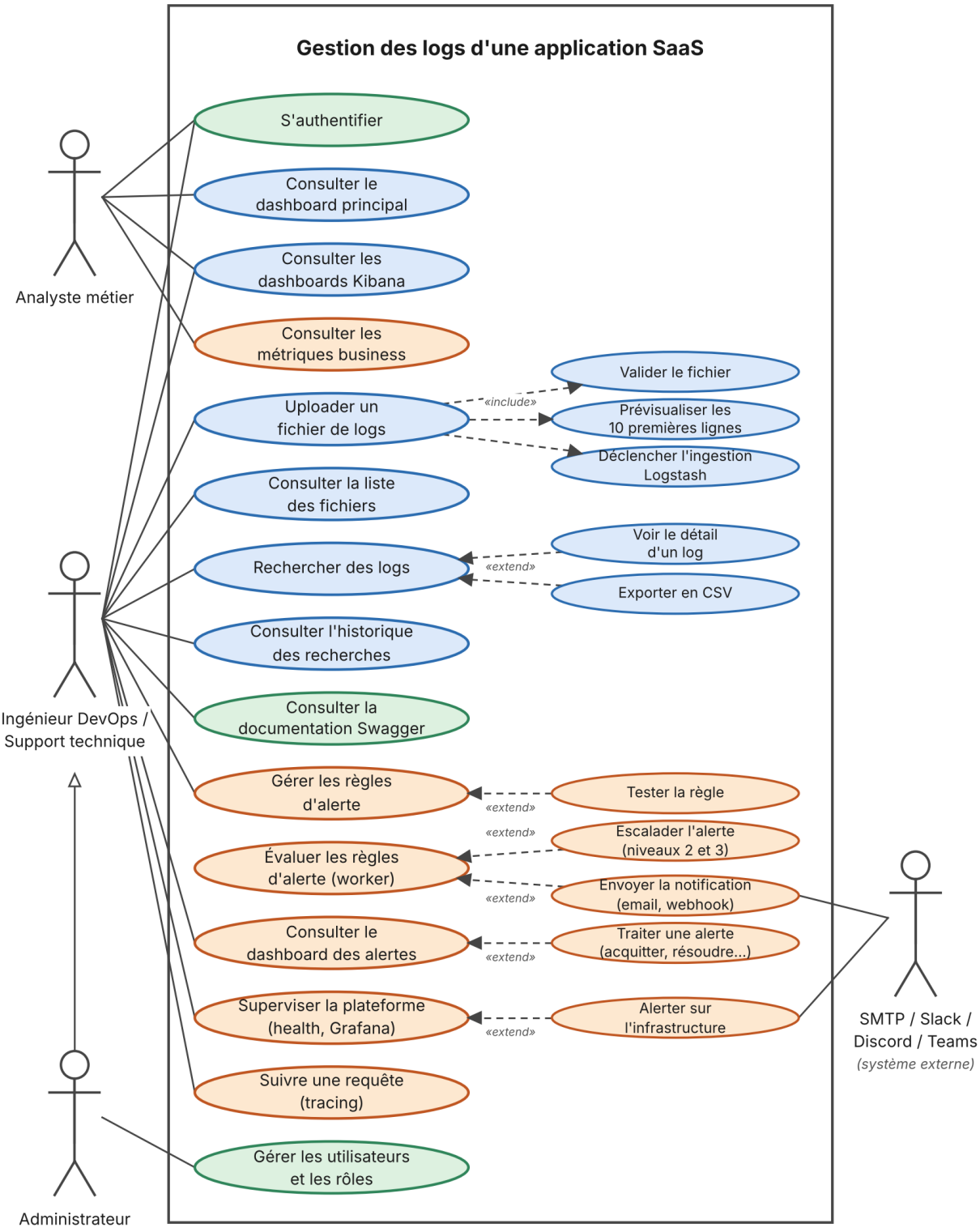


Figure 1 – Diagramme de cas d'utilisation de LogWatch (bleu : niveau obligatoire ; vert : niveau intermédiaire ; orange : niveau avancé)

A.5 User stories priorisées

P1 = niveau obligatoire (15/20) ; P2 = niveau intermédiaire ; P3 = niveau avancé (modules G et L). S2 à S6 = sprint de réalisation (voir le planning). Ces **42 user stories** sont celles du product backlog Jira. Dans le tableau, « DevOps » désigne l'Ingénieur DevOps / Support technique ; « Utilisateur » désigne tout acteur humain authentifié.

ID	Prio.	Sprint	Rôle	Je veux...	Bénéfice
Epic : Gestion des fichiers de logs (Module 1)					
US01	P1	S4	DevOps	uploader un fichier CSV, JSON ou TXT par glisser-déposer, avec une limite de taille configurable et une barre de progression	afin d'injecter des logs sans ligne de commande
US02	P1	S3	DevOps	que le format, la taille et la structure du fichier soient validés, avec un message d'erreur clair	afin de ne pas injecter un fichier invalide
US03	P1	S4	DevOps	prévisualiser les 10 premières lignes avant l'injection	afin de vérifier le contenu du fichier
US04	P1	S3	DevOps	que le fichier validé soit envoyé automatiquement à Logstash et que son statut soit visible (en attente, en cours, indexé, échec)	afin de savoir quand mes logs sont disponibles
US05	P1	S4	DevOps	consulter la liste des fichiers uploadés (nom, date, taille, statut, nombre de logs) avec tri, pagination et filtres par statut et date	afin de retrouver l'historique des uploads
Epic : Configuration de la stack ELK (Module 2)					
US06	P1	S3	DevOps	un index template logs-saas-* avec un mapping adapté au scénario SaaS	afin que chaque champ ait le bon type (date, keyword, text, ip, etc.)
US07	P1	S3	DevOps	que les fichiers CSV soient parsés et indexés par un pipeline Logstash	afin de pouvoir rechercher leurs logs dans Elasticsearch
US08	P1	S3	DevOps	que les fichiers JSON soient parsés et indexés par un second pipeline Logstash, avec gestion des erreurs de parsing	afin d'ingérer les deux formats de logs
US09	P1	S4	Analyste	consulter au moins 3 visualisations Kibana (temps de réponse par endpoint, erreurs 5xx par jour, utilisateurs actifs)	afin de suivre les KPI du SaaS
US10	P1	S4	Analyste	consulter un dashboard Kibana complet, exportable en JSON	afin d'avoir une vue d'ensemble et versionner la configuration
US40	P1	S4	DevOps	identifier les requêtes SQL les plus lentes dans les logs de base de données	afin de savoir lesquelles optimiser
US41	P1	S4	Analyste	voir les features les plus et les moins utilisées	afin d'analyser l'adoption des nouvelles fonctionnalités
Epic : Interface web de base (Module 3)					
US11	P1	S4	Analyste	voir sur la page d'accueil les statistiques globales (logs indexés, logs du jour, logs en erreur, fichiers uploadés) et des graphiques de synthèse	afin d'avoir une vue d'ensemble de la plateforme
US12	P1	S4	DevOps	rechercher en texte libre avec des filtres niveau, service et plage de dates	afin de diagnostiquer un incident rapidement
US13	P1	S4	DevOps	voir les logs trouvés dans un tableau paginé (50 par page) et triable, avec le nombre total de résultats	afin d'analyser rapidement les résultats
US14	P1	S4	DevOps	voir le détail d'un log et exporter les résultats en CSV	afin d'analyser en profondeur et partager les résultats
US15	P1	S4	Analyste	accéder aux dashboards Kibana depuis l'application web	afin de ne pas quitter l'interface LogWatch

ID	Prio.	Sprint	Rôle	Je veux...	Bénéfice
US16	P1	S4	Utilisateur	utiliser l'interface sur ordinateur, tablette et mobile avec un menu de navigation clair	afin de travailler depuis n'importe quel appareil
Epic : Intégration MongoDB (Module 4)					
US17	P1	S3	DevOps	que les métadonnées de chaque fichier soient enregistrées dans MongoDB	afin de conserver l'historique complet des uploads
US18	P1	S4	DevOps	retrouver mes dernières recherches dans une page d'historique	afin de les relancer sans les retaper
Epic : Setup et déploiement Docker (Module 5)					
US19	P1	S2	DevOps	démarrer toute la plateforme avec une seule commande docker compose up	afin d'obtenir un déploiement reproductible
US20	P1	S2	DevOps	configurer la plateforme avec un fichier .env (et un .env.example versionné)	afin de ne jamais exposer de secrets dans Git
Epic : Modules intermédiaires (Auth, Cache, Swagger)					
US21	P2	S3	Utilisateur	m'authentifier avec un compte et une session	afin de protéger l'accès aux données
US22	P2	S3	Administrateur	gérer les utilisateurs et leurs rôles	afin de contrôler les accès à la plateforme
US23	P2	S5	DevOps	que les résultats de recherche soient mis en cache dans Redis	afin d'obtenir des réponses plus rapides
US42	P2	S5	DevOps	limiter le nombre de requêtes API par utilisateur sur une période donnée	afin de protéger l'application contre les abus et les surcharges
US24	P2	S5	DevOps	consulter la documentation Swagger/OpenAPI de l'API REST	afin de tester et comprendre les endpoints
Epic : Module avancé G : Alerting intelligent					
US25	P3	S5	DevOps	créer une règle d'alerte (nom, condition, fenêtre de temps, gravité), par exemple temps de réponse supérieur à 3 s	afin d'être prévenu automatiquement
US26	P3	S5	DevOps	tester une règle avant de l'activer	afin d'éviter les fausses alertes
US27	P3	S5	DevOps	qu'un worker évalue les règles périodiquement, avec silencing et cooldown	afin de ne pas recevoir plusieurs alertes pour le même incident
US28	P3	S5	DevOps	recevoir l'alerte par email (SMTP) avec un lien vers la recherche concernée	afin de réagir sans surveiller l'écran
US29	P3	S5	DevOps	recevoir l'alerte par webhook (Slack, Discord ou Teams) avec retry automatique	afin d'être notifié dans mon outil de travail
US30	P3	S5	DevOps	voir un dashboard des alertes (actives, historique, statistiques, MTTR)	afin de suivre les incidents
US31	P3	S5	DevOps	acquitter, résoudre ou suspendre (snooze) une alerte et la commenter	afin de suivre le traitement des incidents
US32	P3	S6	DevOps	une escalade automatique des alertes non traitées (niveau 2 après 15 min, niveau 3 après 1 h)	afin qu'un incident non traité ne soit pas oublié
Epic : Module avancé L : Observabilité et monitoring					
US33	P3	S6	DevOps	consulter l'état de santé de la plateforme via /health	afin de détecter rapidement un service en panne
US34	P3	S3	DevOps	que l'application produise des logs JSON structurés avec un trace_id	afin d'analyser et corréler les logs facilement
US35	P3	S6	DevOps	que l'application expose ses métriques au format Prometheus (/metrics)	afin de les superviser
US36	P3	S6	DevOps	consulter des dashboards Grafana (application, Elasticsearch, services Docker)	afin de surveiller la santé du système
US37	P3	S6	Analyste	consulter les métriques business du SaaS (MAU, churn rate)	afin de suivre l'usage du produit

ID	Prio.	Sprint	Rôle	Je veux…	Bénéfice
US38	P3	S6	DevOps	recevoir des alertes d'infrastructure (CPU/RAM supérieur à 80 %, disque inférieur à 10 %, service down) par Slack ou email	afin d'intervenir avant la panne
US39	P3	S6	DevOps	suivre une requête à travers les composants avec Jaeger ou Zipkin	afin d'identifier les lenteurs

A.6 Règles métier spécifiques

N°	Règle
RM1	Formats acceptés : CSV, JSON, TXT ; taille maximale 100 Mo par fichier (configurable).
RM2	Un fichier invalide (format, colonnes, syntaxe JSON) n'est jamais injecté ; l'erreur indique la cause précise.
RM3	Niveaux de log : DEBUG, INFO, WARNING, ERROR, CRITICAL. Un « log avec erreur » est de niveau ERROR ou CRITICAL.
RM4	Chaque fichier a un statut : `pending`, `ingesting`, `indexed` ou `failed`, avec son nombre de logs.
RM5	Les résultats de recherche sont paginés par 50 et triables par date, niveau et service.
RM6	Toute recherche est enregistrée dans l'historique de l'utilisateur.
RM7	Les secrets (mots de passe, clés) sont dans le fichier `.env`, jamais dans Git ; seul `.env.example` est versionné.
RM8	*(Intermédiaire)* Un utilisateur non connecté n'accède à aucune page, sauf la page de connexion. Les rôles sont : DevOps, Analyste, Administrateur.
RM9	*(Intermédiaire)* Les résultats de recherche sont mis en cache dans Redis pendant 60 secondes ; le nombre de requêtes API par utilisateur est limité (rate limiting).
RM10	*(Avancé, module G)* Une règle d'alerte a une gravité (Low, Medium, High, Critical) et une fenêtre de temps (5 min, 15 min, 1 h, 24 h). Exemple du scénario : alerte si le temps de réponse moyen dépasse 3000 ms.
RM11	*(Avancé, module G)* Un délai de cooldown évite d'envoyer plusieurs fois la même alerte pour un incident identique.
RM12	*(Avancé, module G)* Une alerte peut être acquittée, résolue ou suspendue (snooze) et commentée ; un webhook en échec est relancé automatiquement.
RM13	*(Avancé, module G)* Une alerte non traitée est automatiquement escaladée au niveau 2 après 15 minutes, puis au niveau 3 après 1 heure.

A.7 Product backlog et organisation Scrum (Jira)

Le product backlog est géré dans Jira (espace **LogWatch**, clé `LW`) : 11 epics, les user stories du tableau A.5 et leurs tâches techniques, réparties en 6 sprints d'une semaine du 26 octobre au 5 décembre 2026. Chaque story porte sa priorité (P1, P2, P3) et son sprint de réalisation.

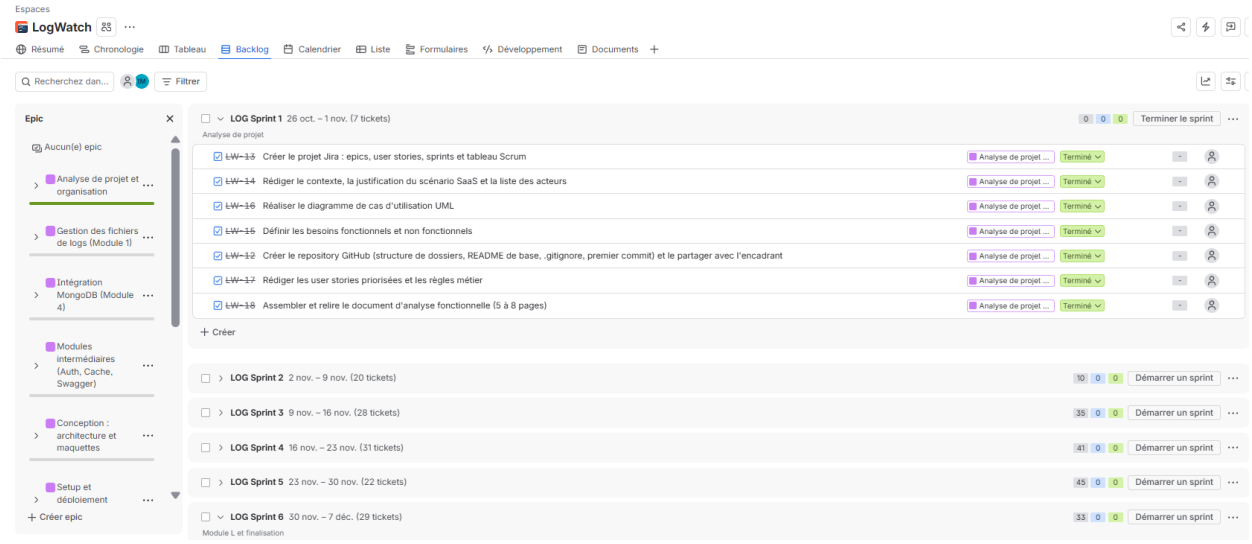


FIGURE 2 – Product backlog Jira : epics et sprints

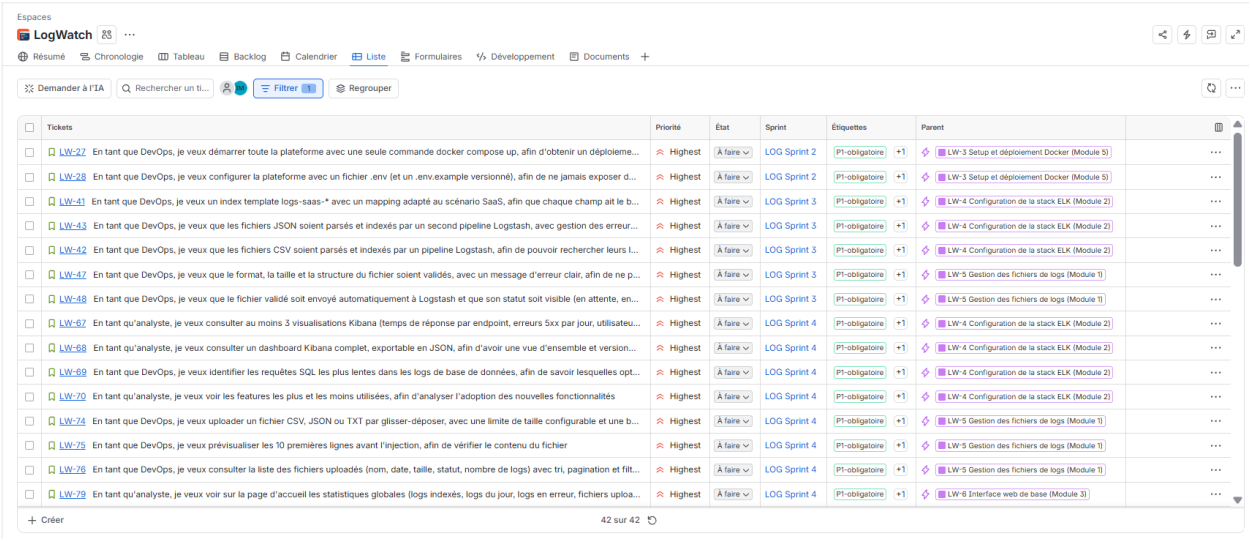


FIGURE 3 – User stories dans Jira : epic, priorité et sprint

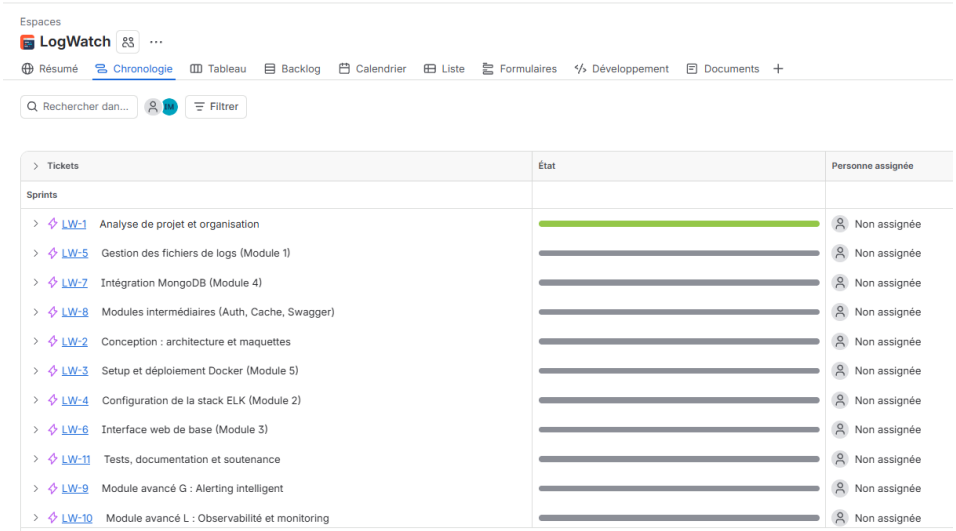


FIGURE 4 – Chronologie Jira : epics répartis sur les 6 sprints